TECHNICAL SPECIFICATION
Personal AI Desktop Assistant

ชื่อชั่วคราว: Bo AI

Version 1.0 | วันที่จัดทำ: 15 ตุลาคม 2026

เอกสารพร้อมส่งให้ทีมพัฒนา / Antigravity ใช้งานได้ทันที

1. ภาพรวมโครงการ (Project Overview)

ชื่อโครงการ: Personal AI Desktop Assistant (ชื่อชั่วคราว: Bo AI)

วัตถุประสงค์: สร้างแอปพลิเคชัน Desktop บน Windows ที่ทำหน้าที่เป็นผู้ช่วย AI ส่วนตัว สามารถรอฟังคำเรียก (Wake Word) ตลอดเวลา รับคำสั่งด้วยเสียง ประมวลผลด้วย LLM แล้วพูดตอบกลับ พร้อมให้ผู้ใช้ปรับบุคลิก ความรู้ ความจำ และเครื่องมือได้เอง ทำงานในพื้นหลัง (System Tray) และเปิดอัตโนมัติเมื่อเข้าสู่ระบบ Windows

กลุ่มผู้ใช้เป้าหมาย: ผู้ใช้ทั่วไปที่ต้องการ AI ผู้ช่วยส่วนตัวที่ปรับแต่งได้สูง และควบคุมข้อมูลได้เอง

แพลตฟอร์มหลัก: Windows 10/11 (x64) — รองรับ macOS/Linux ในอนาคต

2. เป้าหมายและขอบเขต (Goals & Scope)

2.1 เป้าหมายหลัก

- มีระบบ Always-on Listening ที่ใช้ Wake Word
- สนทนาด้วยเสียงภาษาไทยได้ธรรมชาติ
- ผู้ใช้สามารถฝึกและปรับแต่ง AI ได้ผ่าน Training Studio
- เก็บข้อมูลส่วนตัวในเครื่องเป็นหลัก (Privacy-first)
- มีสถาปัตยกรรมที่ขยายได้ในอนาคต

2.2 ขอบเขตเฟสแรก (MVP)

- Desktop App + System Tray
- เลือกไมค์ / ลำโพง ได้เอง
- Wake Word Detection
- สนทนาด้วยเสียงครบวงจร (STT → LLM → TTS)
- หน้า Conversation + Memory พื้นฐาน
- หน้า Training Studio ขั้นต้น

2.3 สิ่งที่อยู่นอกขอบเขตเฟสแรก

- Multi-device sync แบบเต็มรูปแบบ

- Local LLM แบบรันในเครื่อง (จะเพิ่มภายหลัง)
- Mobile Application
- การควบคุมฮาร์ดแวร์ขั้นสูง (IoT)

3. สถาปัตยกรรมระบบ (System Architecture)

3.1 ภาพรวมสถาปัตยกรรม

ระบบแบ่งเป็น 3 ชั้นหลัก: Electron Main Process (จัดการหน้าต่าง, Tray, Auto-launch), Renderer Process (React UI), และ Local Services (Node.js) ที่ดูแลเสียง, AI, และฐานข้อมูล

Electron Main Process
• System Tray • Auto Launch • Window Management

Renderer Process (React + TypeScript)
• Home • Conversation • Voice & Devices • Training Studio
• Memory • Tools • Settings

Local Services (Node.js)
• Audio Service (Mic, Speaker, Wake Word, STT/TTS)
• AI Service (LLM, Memory, Personality, Tools)
• Database Service (SQLite)

3.2 หลักการออกแบบ

- Privacy-first — ข้อมูลส่วนตัวเก็บในเครื่องเป็นหลัก
- Modular — แยก Service ชัดเจน สามารถเปลี่ยน LLM หรือ STT/TTS ได้ง่าย
- Event-driven — ใช้ Event Emitter ระหว่าง Service
- Fail-safe — หาก Service ใดล่ม ระบบต้องกู้คืนได้

4. Technology Stack

หมวด	เทคโนโลยี	หมายเหตุ
Desktop Framework	Electron 33+	
Frontend	React 18+ + TypeScript	
UI Library	Tailwind CSS + shadcn/ui	
State Management	Zustand	
Backend Runtime	Node.js 20 LTS	
Database	better-sqlite3	เบาและเร็ว
Wake Word	Picovoice Porcupine / OpenWakeWord	รันออฟไลน์ได้
Speech-to-Text	OpenAI Whisper (API/Local)	รองรับภาษาไทยดี
Text-to-Speech	Edge TTS / Azure Neural TTS	เสียงไทยธรรมชาติ
LLM	GPT-4o / Claude / Gemini	เลือกได้ผ่าน Settings
Build & Package	Vite + electron-builder	สร้าง .exe installer

5. โครงสร้างโปรเจกต์ (Project Structure)

```
bo-ai/
 electron/            # Main process
   main.ts
   preload.ts
   tray.ts
   ipc-handlers.ts
 src/               # React Frontend
   pages/
     Home.tsx
     Conversation.tsx
     VoiceDevices.tsx
     TrainingStudio/
     Memory.tsx
     Tools.tsx
     Settings.tsx
   components/
   stores/
   types/
 services/          # Node.js Services
   audio/
   ai/
   db/
 data/
   personality/
   knowledge/
   memory.db
 package.json
```

6. รายละเอียดฟีเจอร์ (Feature Specification)

6.1 ระบบเสียง (Audio System)

6.1.1 การเลือกอุปกรณ์

- ดึงรายชื่อ Input Devices และ Output Devices จากระบบ
- ผู้ใช้เลือกไมโครโฟนและลำโพงได้
- บันทึกการตั้งค่าลงฐานข้อมูล
- ปุ่มทดสอบไมค์ (แสดงระดับเสียง real-time)
- ปุ่มทดสอบลำโพง (เล่นเสียงตัวอย่าง)

6.1.2 Wake Word Detection

- รองรับชื่อเรียกหลัก + ชื่อสำรอง (สูงสุด 3 ชื่อ)
- ปรับ Sensitivity ได้ (0–100)
- เมื่อตรวจจับสำเร็จ → เปลี่ยนสถานะเป็น listening
- หลังจากจบสนทนา → กลับสู่สถานะ idle
- รองรับโหมด Always Listening

6.1.3 Speech Pipeline

1. ตรวจจับ Wake Word
2. เริ่มบันทึกเสียงคำสั่ง (มี timeout และ VAD)
3. ส่งไป Speech-to-Text

4. ส่งข้อความไป LLM พร้อม context
5. รับคำตอบ → Text-to-Speech
6. เล่นเสียงตอบกลับ
7. กลับสู่สถานะรอคำเรียก

6.2 ระบบสนทนา (Conversation)

- แสดงประวัติสนทนาแบบ real-time
- เก็บ Context ล่าสุด (กำหนดจำนวนข้อความได้)
- รองรับการลบประวัติรายการหรือทั้งหมด
- แสดงสถานะ AI ชัดเจน: Idle / Listening / Thinking / Speaking

6.3 Training Studio

โมดูล	รายละเอียด
Personality	กำหนดชื่อ, บุคลิก, สไตล์การพูด, ระดับความเป็นทางการ, System Prompt
Knowledge Base	อัปโหลดไฟล์ (txt, md, pdf) หรือใส่ข้อความ, รองรับการค้นคืนแบบง่าย
Example Training	ใส่คู่คำถาม-คำตอบตัวอย่าง เพื่อปรับพฤติกรรม
Test Lab	ทดสอบการตอบโดยพิมพ์ข้อความ (ไม่ใช้เสียง)

6.4 Memory System

- เก็บความจำระยะสั้น (Conversation Context)
- เก็บความจำระยะยาว (ข้อเท็จจริงเกี่ยวกับผู้ใช้)
- ผู้ใช้สามารถดู / แก้ไข / ลบความจำได้
- มีระบบจัดลำดับความสำคัญของความจำ

6.5 Tools & Integrations (เฟสถัดไป)

- สภาพอากาศ
- ควบคุมเพลง
- เปิดเว็บไซต์ / ค้นหาข้อมูล
- ตั้งการแจ้งเตือน
- คำสั่งระบบ (ต้องมี confirmation)

7. ฐานข้อมูล (Database Schema)

ใช้ SQLite ผ่าน better-sqlite3

ตาราง settings

```
key TEXT PRIMARY KEY, value TEXT NOT NULL, updated_at DATETIME
```

ตาราง conversations

```
id TEXT PRIMARY KEY, title TEXT, created_at DATETIME, updated_at DATETIME
```

ตาราง messages

```
id TEXT PRIMARY KEY, conversation_id TEXT, role TEXT ('user'|'assistant'|'system'), content TEXT, created_at DATETIME
```

ตาราง memories

```
id TEXT PRIMARY KEY, content TEXT, category TEXT, importance INTEGER, created_at DATETIME, updated_at DATETIME
```

ตาราง knowledge_items

```
id TEXT PRIMARY KEY, title TEXT, content TEXT, source TEXT ('manual'|'file'), file_path TEXT, created_at DATETIME
```

ตาราง personality

```
id INTEGER PRIMARY KEY CHECK (id = 1), name TEXT, system_prompt TEXT, speaking_style TEXT, formality_level INTEGER, updated_at DATETIME
```

8. Internal API / Service Interface

8.1 Audio Service (สรุป Interface)

```
getInputDevices() → Promise<AudioDevice[]>
getOutputDevices() → Promise<AudioDevice[]>
setInputDevice(deviceId) / setOutputDevice(deviceId)
startWakeWordDetection() / stopWakeWordDetection()
onWakeWordDetected(callback)
startRecording() / stopRecording() → Promise<Buffer>
speak(text: string) → Promise<void>
getMicrophoneLevel() → number
```

8.2 AI Service (สรุป Interface)

```
chat(messages: Message[], options?) → Promise<string>
updatePersonality(config: PersonalityConfig)
addMemory(content, category?) / searchMemory(query)
executeTool(toolName, params) → Promise<any>
```

8.3 State Machine หลัก

idle → (Wake Word) → listening → (STT สำเร็จ) → thinking → (LLM ตอบ) → speaking → idle

9. หน้าจอและ UI Specification

หน้า	องค์ประกอบหลัก
Home	สถานะ AI ขนาดใหญ่, ปุ่มเปิด/ปิดการฟัง, ข้อมูลสรุป
Conversation	รายการข้อความ, ช่องพิมพ์สำรอง, ปุ่มล้างประวัติ
Voice & Devices	Dropdown เลือกไมค์/ลำโพง, Slider ความไว, ปุ่มทดสอบ
Training Studio	Tab: Personality / Knowledge / Examples / Test Lab
Memory	รายการความจำ, ค้นหา, แก้ไข, ลบ
Tools	รายการเครื่องมือที่เปิดใช้ได้ + การตั้งค่า
Settings	API Key, โมเดล LLM, ภาษา, Auto-start, Privacy

หลักการ UI: Dark Mode เป็นค่าเริ่มต้น • แสดงสถานะ AI ให้ชัดเจนตลอดเวลา • รองรับทั้งเมาส์และคีย์บอร์ด

10. ความปลอดภัยและความเป็นส่วนตัว

- API Key เก็บแบบเข้ารหัสในเครื่อง (ใช้ safeStorage ของ Electron)
- ข้อมูลสนทนาและความจำเก็บในเครื่องเป็นค่าเริ่มต้น

- คำสั่งที่มีผลต่อระบบ (ปิดเครื่อง, ลบไฟล์ ฯลฯ) ต้องมี confirmation
- ไมโครโฟนทำงานเฉพาะเมื่ออยู่ในโหมดที่ผู้ใช้เปิด
- มีตัวเลือกให้ลบข้อมูลทั้งหมดได้ง่าย
- ไม่ส่งเสียงดิบไปยัง Cloud ยกเว้นตอนทำ STT (และแจ้งผู้ใช้ชัดเจน)

11. แผนการพัฒนา (Development Phases)

Phase	ชื่อ	ระยะเวลา	Deliverable หลัก
1	Foundation	1–2 สัปดาห์	แอปเปิดได้ + System Tray + หน้าพื้นฐาน
2	Audio System	1–2 สัปดาห์	เลือกไมค์/ลำโพง + ทดสอบเสียงได้
3	Wake Word	1–2 สัปดาห์	เรียกชื่อแล้วระบบตอบสนอง
4	AI Conversation	2 สัปดาห์	สนทนาด้วยเสียงได้ครบวงจร
5	Training Studio	2 สัปดาห์	ปรับบุคลิกและความรู้ได้
6	Tools	1–2 สัปดาห์	เครื่องมือพื้นฐานทำงานได้
7	Always-on & Reliability	1 สัปดาห์	ทำงานพื้นหลังเสถียร
8	Polish & Packaging	1 สัปดาห์	Installer + UI ปรับปรุง

12. Non-functional Requirements

- Performance: Wake Word detection latency < 1 วินาที
- Reliability: ระบบทำงานต่อเนื่องได้ ≥ 8 ชั่วโมง โดยไม่ต้องรีสตาร์ท
- Usability: ผู้ใช้ทั่วไปตั้งค่าไมค์และชื่อเรียกได้ภายใน 5 นาที
- Privacy: ข้อมูลส่วนตัวไม่ถูกส่งออกโดยไม่ได้รับอนุญาต
- Extensibility: สามารถเพิ่ม Tool ใหม่ได้โดยไม่กระทบแกนหลัก

13. ความเสี่ยงและแนวทางจัดการ

ความเสี่ยง	ผลกระทบ	แนวทางจัดการ
Wake Word เรียกผิดบ่อย	สูง	ปรับ Sensitivity + มี blacklist คำ
STT ภาษาไทยไม่แม่น	สูง	ทดสอบหลายตัวเลือก (Whisper, Azure, Google)
ใช้ทรัพยากรเครื่องสูง	กลาง	Optimize การทำงานพื้นหลัง
API Cost สูง	กลาง	มีตัวเลือก Local Model ในอนาคต
ความเสถียรของ Electron	กลาง	ทำ Auto-restart + Logging

14. ภาคผนวก

14.1 คำถามที่ต้องตัดสินใจก่อนเริ่มพัฒนา

1. ชื่อเรียก AI ที่ต้องการใช้คืออะไร?
2. LLM ตัวหลักที่จะใช้ในเฟสแรก? (OpenAI / Claude / Gemini)
3. ความจำเก็บเฉพาะในเครื่อง หรือผสม Cloud?

4. ต้องการรองรับ Local LLM ในเฟสใด?

14.2 เอกสารที่เกี่ยวข้อง (ที่จะจัดทำเพิ่ม)

- UI Mockup / Wireframe
- API Key Management Guide
- Wake Word Training Guide (กรณีใช้ Porcupine)
- Database Migration Scripts

เอกสารนี้พร้อมสำหรับส่งให้ Antigravity หรือทีมพัฒนาใช้เป็นข้อกำหนดทางเทคนิคได้ทันที

จัดทำโดย Grok • 15 ตุลาคม 2026